

Índice

Entrega final - Curso Java Back-End - Talento Tech.....	2
Introducción.....	2
Configuración Inicial de Spring Boot.....	2
Desarrollo de Endpoints en Memoria y Pruebas con Thunder Client.....	4
Integración de Persistencia Relacional con Spring Data JPA y MySQL.....	6
1. Configuración del Entorno y Dependencias.....	6
2. Mapeo Objeto-Relacional (ORM) en Entidades.....	7
3. Abstracción de Datos Mediante el Patrón Repository.....	8
4. Verificación de Persistencia en phpMyAdmin.....	9
5. Modelado de Relaciones y Normalización de Datos (@ManyToOne).....	11
6. Ampliación de la Capa de Consultas y Endpoints Avanzados (RESTful API).....	12
Validación de Datos en el Modelo de Dominio.....	13
Arquitectura y Manejo Global de Errores.....	15
Módulo de Gestión de Pedidos y Carrito de Compras.....	17
1. Modelo de Datos y Relaciones (ORM).....	17
2. Lógica de Negocio y Robustez del Sistema.....	18
3. Arquitectura de Control de Errores Centralizada.....	19
Catálogo de Endpoints de la API REST (Diccionario de Rutas).....	19
Módulo de Categorías (/categorias).....	19
Módulo de Productos (/productos).....	20
Módulo de Pedidos (/pedidos).....	21
Conclusiones Finales.....	22
Anexo.....	24
Modificación del Modelo de Datos por Requerimientos de Interfaz (Frontend).....	24

Entrega final - Curso Java Back-End - Talento Tech

Introducción

El presente documento detalla el proceso evolutivo de desarrollo de la aplicación de comercio electrónico (E-commerce). Originalmente, el sistema fue concebido como una aplicación monolítica de consola en Java puro, donde el flujo de interacción con el usuario y el control del negocio estaban centralizados en menús interactivos por comandos.

Con el avance de los requerimientos técnicos y la necesidad de escalar el software hacia estándares profesionales de la industria, se determinó realizar una migración completa hacia el framework Spring Boot. Este cambio tecnológico permite desacoplar la interfaz de usuario de la lógica de negocio mediante una arquitectura basada en servicios web y APIs REST, facilitando la persistencia de datos, el manejo eficiente de dependencias y la integración con interfaces gráficas web o móviles.

Configuración Inicial de Spring Boot

Para realizar la migración del sistema, se utilizó la herramienta *Spring Initializr* con el objetivo de generar una arquitectura base sólida y estandarizada bajo las convenciones de la industria. Las especificaciones técnicas seleccionadas para el entorno de ejecución fueron:

- **Gestor de Proyectos:** Maven, para centralizar el ciclo de vida y la descarga automatizada de librerías.
- **Lenguaje y Versión:** Java 17, asegurando el uso de características modernas de soporte a largo plazo (LTS).
- **Versión de Framework:** Spring Boot 4.1.0, garantizando estabilidad en los componentes web.
- **Metadatos del Proyecto:** Organizado bajo el paquete raíz `com.techlab.ecommerce`.
- **Dependencias Clave:** Se incorporó el módulo **Spring Web**, el cual provee el servidor embebido Apache Tomcat y las herramientas de Spring necesarias para construir la arquitectura RESTful.

Una vez importada la estructura en el entorno de desarrollo, se reemplazaron por completo los antiguos componentes interactivos de consola (**MenuProducto** y **Validador**) por una división lógica limpia distribuida en paquetes especializados:

1. **Capa de Modelos (model):** Contiene las clases de datos puros como **Producto**. Se diseñó de forma aislada, lo que significa que el modelo solo encapsula sus propios atributos y métodos de acceso (getters y setters), delegando cualquier lógica operativa externa.
2. **Capa de Servicios (service):** Centralizada en **ProductoService**, componente que asume la responsabilidad total de gestionar el estado del inventario y las reglas de negocio.

Mecanismo de Inicialización y Carga de Datos de Prueba

Dado que el sistema, en esta etapa inicial, aún no implementa una base de datos física persistente, se requería una estrategia para que la aplicación iniciara con información disponible de manera inmediata para los clientes web. Para resolver esto, en la clase principal **EcommerceApplication** se configuró un método inicializador mediante la anotación **@Bean** de Spring.

Este método devuelve un objeto de tipo **CommandLineRunner**, una interfaz provista por el framework cuya única función es ejecutar bloques de código de forma automática justo un instante después de que el servidor web finaliza su arranque. Mediante este disparador, se inyecta el **ProductoService** y se ejecutan de manera transparente los métodos para poblar la lista interna con un catálogo inicial de productos. De este modo, se garantiza que los datos estén completamente cargados en memoria antes de que el sistema empiece a recibir las primeras peticiones HTTP externas.

```
@Bean
public CommandLineRunner cargarDatos(ProductoService service){
    return args -> {
        service.guardarProducto(new Producto(nombre: "Redmi Note 10",
        precio: 250000, stock: 5, categoria: "Celular", marca: "Xiaomi"));
        service.guardarProducto(new Producto(nombre: "Galaxy A16 4G",
        precio: 399500, stock: 3, categoria: "Celular", marca: "Samsung"));
        service.guardarProducto(new Producto(nombre: "Jd Capri 1.83", precio: 40500,
        stock: 3, categoria: "Smartwatch", marca: "JD"));
        service.guardarProducto(new Producto(nombre: "Gamer G435", precio: 126000,
        stock: 10, categoria: "Auriculares", marca: "Logitech G"));
        service.guardarProducto(new Producto(nombre: "Ecotank L1250",
        precio: 335699, stock: 2, categoria: "Impresora", marca: "Epson"));
    };
}
```

Nota técnica sobre `@Bean` y `CommandLineRunner`

La anotación `@Bean` le indica a Spring Boot que registre de forma explícita el objeto devuelto por el método en su contenedor interno de dependencias.

Al ser de tipo `CommandLineRunner`, el framework garantiza su ejecución automática en el instante posterior al inicio del servidor web, logrando poblar la lista de productos antes de recibir tráfico.

Se creó el paquete `com.techlab.ecommerce.controller` y se programó la clase `ProductoController`. Este componente actúa como el nuevo "punto de entrada" web del sistema, sustituyendo la lógica de consola por el protocolo HTTP.

El controlador hace uso estratégico de la Inyección de Dependencias por Constructor, permitiendo que Spring Boot suministre de forma transparente la instancia de `ProductoService` requerida, evitando el acoplamiento rígido que provocaría un operador `new` tradicional.

```
@RestController
@RequestMapping("/productos")
public class ProductoController {
    private final ProductoService service;

    public ProductoController(ProductoService service){
        this.service = service;
    }

    @GetMapping
    public List<Producto> listarProductos(){
        return service.getListaProductos();
    }
}
```

Con esta implementación, la consulta a la URL `http://localhost:8080/productos` expone la colección de productos de forma nativa en formato de texto JSON, concluyendo con éxito la transición hacia el modelo cliente-servidor.

Desarrollo de Endpoints en Memoria y Pruebas con Thunder Client

Una vez consolidada la estructura base del proyecto en Spring Boot, el siguiente paso consistió en trasladar toda la lógica operativa que previamente corría por consola hacia controladores web funcionales. En esta etapa, el almacenamiento de la información se mantiene de forma temporal **en memoria** a través de las colecciones administradas por el servicio, sirviendo como paso previo y fundamental antes de la futura integración con una base de datos relacional.

Para este desarrollo, se optó por una arquitectura homogénea y profesional implementando la clase `ResponseEntity` en todos los métodos del controlador. La inclusión de esta herramienta es clave, ya que permite controlar manualmente el componente más importante de una comunicación web: *el código de estado HTTP*.

Detalle del Proceso Lógico Implementado:

- **Estandarización de Respuestas:** Al envolver los retornos en un `ResponseEntity`, el sistema no solo envía los datos en formato JSON, sino que además le responde al cliente con códigos semánticos precisos. Por ejemplo, al crear un recurso se retorna de forma explícita un estado `201 Created` en lugar del clásico `200 OK`, lo cual es una buena práctica de diseño de APIs REST.
- **Control de Flujo y Excepciones:** Se integraron bloques `try-catch` dentro de los endpoints de búsqueda (`GET /{id}`), actualización (`PUT`) y eliminación (`DELETE`). Esto se hizo con el fin de interceptar la excepción personalizada `ProductoNoEncontradoException` que ya se utilizaba previamente en la lógica de negocio. Si el producto solicitado no existe, el controlador detiene el flujo regular y genera limpiamente una respuesta `404 Not Found`, evitando que la aplicación web falle o exponga errores internos de Java al usuario.

Validación del Ciclo de Vida del Request mediante Thunder Client

Para asegurar que cada método HTTP respondiera exactamente a lo esperado, se incorporó al entorno de desarrollo la extensión *Thunder Client* dentro de Visual Studio Code. Esta herramienta permitió simular peticiones reales de un cliente (como si fuera el Front-End de la aplicación) para testear el comportamiento del servidor localmente en `http://localhost:8080/productos`. Gracias a la bitácora de actividad de Thunder Client, se pudo verificar visualmente que tanto el envío de datos en el cuerpo de las peticiones (`@RequestBody`) como los parámetros de ruta (`@PathVariable`) se procesan e impactan correctamente en el estado del servicio.

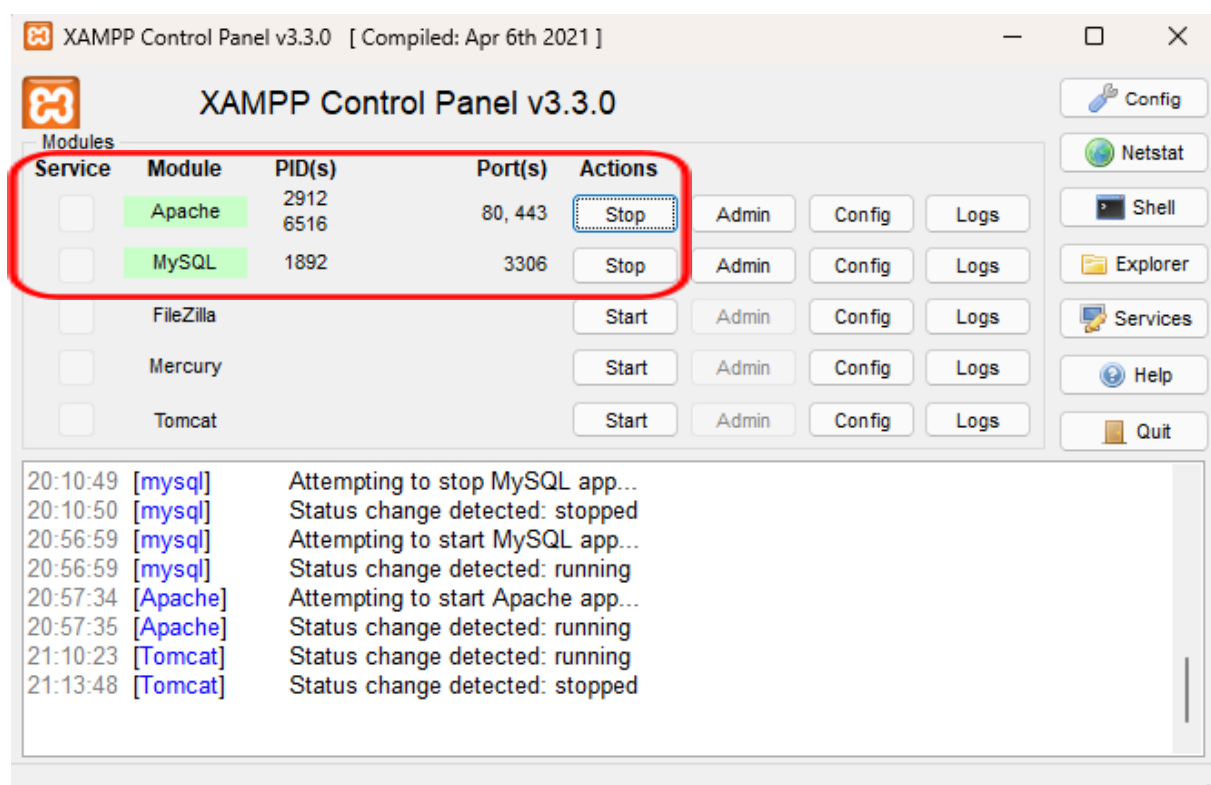
Ampliación de la Arquitectura (Módulo de Categorías)

Como demostración de la escalabilidad del diseño adoptado, se replicó exactamente esta misma arquitectura por capas (`model`, `service` y `controller`) para la gestión de las **Categorías** del e-commerce. Siguiendo el estándar fijado, este nuevo módulo opera de forma independiente en

memoria, cuenta con sus propias excepciones de negocio (como `CategoriaNoEncontradaException`) y expone un CRUD estandarizado bajo la ruta `/categorias`. Las peticiones de alta, baja y modificación de este recurso también fueron testeadas exhaustivamente a través de *Thunder Client*, respondiendo de manera consistente con los códigos de estado HTTP correspondientes.

Integración de Persistencia Relacional con Spring Data JPA y MySQL

Con el objetivo de otorgar persistencia real a los datos y sustituir el almacenamiento temporal en memoria, se implementó una base de datos relacional dentro de la arquitectura del e-commerce. Para el entorno de desarrollo local, se utilizó la suite **XAMPP**, ejecutando su módulo de MySQL bajo el puerto estándar 3306 y administrando el esquema mediante la interfaz gráfica web **phpMyAdmin**.



1. Configuración del Entorno y Dependencias

El proceso comenzó con la edición del archivo de automatización `pom.xml` de Maven, incorporando dos librerías críticas de Spring Boot que habilitan la comunicación con el motor de base de datos:

- **Spring Data JPA (spring-boot-starter-data-jpa):** Módulo encargado de gestionar el mapeo objeto-relacional (ORM) mediante Hibernate, permitiendo interactuar con las tablas usando objetos de Java en lugar de sentencias SQL nativas manuales.
- **MySQL Driver (mysql-connector-j):** Controlador oficial que actúa como puente de comunicación física entre la máquina virtual de Java y el servidor MySQL.

Seguidamente, se parametrizó la conexión dentro del archivo centralizado de configuración `application.properties`:

```
spring.application.name=ecommerce
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Nota Técnica: La propiedad `ddl-auto=update`

Es una directiva estratégica de Hibernate. Su función es inspeccionar las clases Java y, en cada arranque del servidor Apache Tomcat, alterar o crear de manera automática las estructuras de las tablas correspondientes en MySQL sin borrar los registros preexistentes. Las propiedades de visualización SQL habilitan el monitoreo en tiempo real por consola de las consultas generadas por el framework.

2. Mapeo Objeto-Relacional (ORM) en Entidades

Para que el motor JPA reconozca las clases de negocio como tablas de base de datos, se refactorizaron las entidades del modelo (Producto y Categoría) mediante anotaciones estándar de persistencia de Jakarta:

- **@Entity y @Table(name = "..."):** @Entity le indica al framework que la clase representa una entidad persistente. Se complementó con @Table para definir de manera explícita el nombre que adoptará la tabla física dentro del esquema de MySQL, aislando el diseño de la base de datos de las convenciones de nombres de Java.

- **@Id** y **@GeneratedValue(strategy = GenerationType.IDENTITY)**: Configuran la clave primaria del registro delegando la responsabilidad del autoincremento secuencial de manera directa al motor relacional de MySQL. Esto permite una gestión eficiente de los índices y reemplaza los contadores manuales estáticos (`counterId`) usados en la etapa previa en memoria.
- **@Column(name = "...", nullable = false, length = ...)**: Se utilizó para mapear detalladamente cada atributo con su columna correspondiente. La propiedad `nullable = false` actúa como una restricción a nivel de base de datos, impidiendo el registro de campos obligatorios vacíos (como el nombre, precio o stock). El atributo `length` acota la longitud máxima de almacenamiento de cadenas de texto (por ejemplo, limitando el nombre a 100 caracteres y la categoría a 50), optimizando el uso del espacio físico en el disco del servidor.

3. Abstracción de Datos Mediante el Patrón Repository

Con el propósito de mantener el desacoplamiento arquitectónico y respetar el principio de responsabilidad única, se incorporó el **Patrón Repository**. Este patrón actúa como una capa divisoria entre la lógica de negocio pura de los servicios y los accesos de bajo nivel a las tablas.

Para lograrlo, se diseñaron las interfaces `ProductoRepository` y `CategoriaRepository` heredando directamente de la interfaz genérica `JpaRepository<T, ID>`. Esta herencia le otorga al proyecto, de manera implícita y automatizada, la implementación de todas las operaciones transaccionales del CRUD tradicional sin necesidad de que el desarrollador deba tipear sentencias SQL manuales en los servicios:

- **save(entidad)**: Resuelve mediante la misma instrucción tanto la inserción de nuevos registros como la actualización de entidades persistentes preexistentes.
- **findAll()** y **findById(id)**: Facilitan la lectura completa o unitaria de los recursos mapeados. Cabe destacar que `findById` devuelve un objeto contenedor de tipo `Optional`, lo que obliga a implementar flujos defensivos eficientes para mitigar errores catastróficos de tipo puntero nulo (`NullPointerException`).
- **delete(entidad)**: Ejecuta de forma directa la remoción física del registro correspondiente de la tabla MySQL.

Consecuentemente, los componentes de servicios (`ProductoService` y `CategoriaService`) se refactorizaron por completo para utilizar inyección de dependencias basada en constructores. Se removieron por completo las colecciones temporales de tipo `ArrayList` y se reemplazó el control de flujo con el uso moderno de expresiones lambda (`.orElseThrow()`). De esta manera, si un endpoint solicita un recurso mediante un identificador que no existe físicamente en las tablas, el servicio lanza de forma controlada sus respectivas excepciones de negocio (`ProductoNoEncontradoException` o `CategoriaNoEncontradaException`), garantizando un comportamiento predecible del backend.

4. Verificación de Persistencia en phpMyAdmin

Una vez levantada la aplicación con la nueva configuración, el motor ORM procesó con éxito el mapeo, traduciendo de forma transparente los atributos Java a columnas relacionales. La correcta inicialización de la base de datos `ecommerce` y la generación automática de su estructura se validó visualmente mediante el gestor del servidor local.

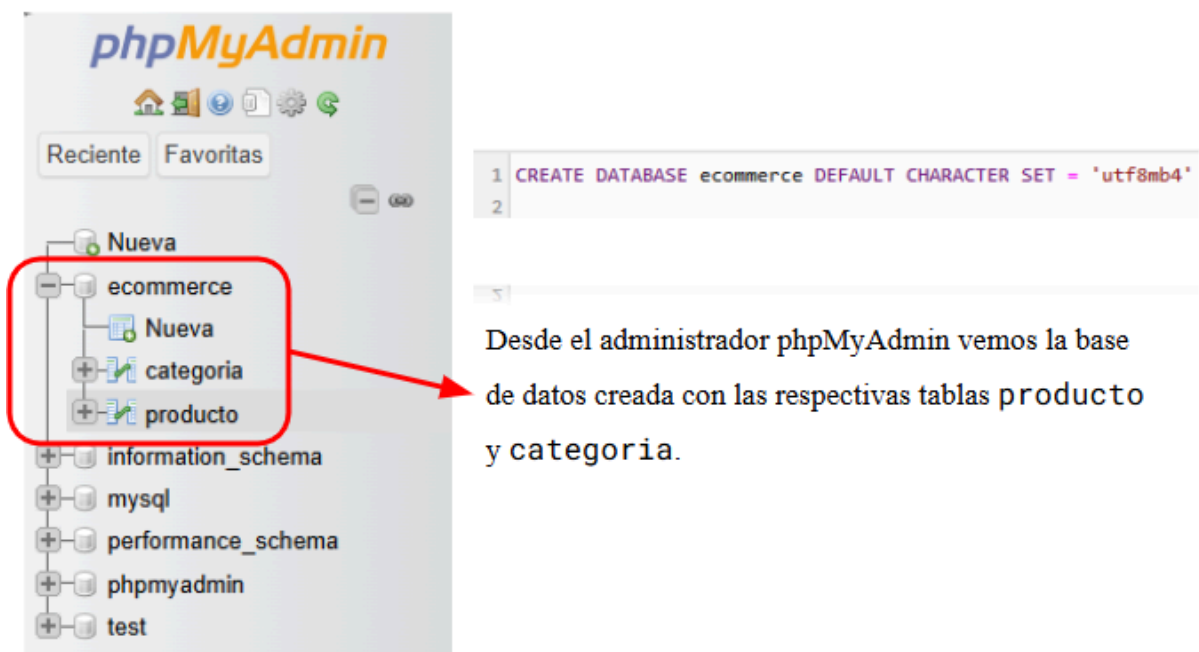


Tabla categoria:

```
SELECT * FROM `categoria`
```

id descripcion nombre

Aquí vemos los atributos de la tabla *categoria*, los cuales son *id*, *descripcion* y *nombre*. Estos campos se corresponden directamente con los atributos del objeto en Java.

Tabla producto:

```
SELECT * FROM `producto`
```

id categoria marca nombre precio stock

Aquí vemos los atributos de la tabla *producto*, los cuales son *id*, *categoria*, *nombre*, *precio*, *marca* y *stock*. Estos campos se corresponden directamente con los atributos del objeto en Java.

La exitosa convergencia entre la lógica de negocio desarrollada en Spring Boot y el servidor local de base de datos se validó mediante la realización de pruebas de integración extremo a extremo. Al enviar peticiones HTTP a través del cliente de pruebas *Thunder Client*, el backend interactuó de forma transparente con los repositorios y persistió los cambios físicamente.

Al auditar el estado del esquema relacional desde el panel de control web de phpMyAdmin, se constató la consistencia de los tipos de datos parametrizados. Asimismo, se comprobó que los registros enviados por el cliente se almacenaron correctamente, reflejando fielmente las restricciones impuestas por las anotaciones del código fuente:

						id	categoria	marca	nombre	precio	stock
☐	 Editar	 Copiar	 Borrar		1	Celular	Xiaomi	Redmi Note 10	250000	5	
☐	 Editar	 Copiar	 Borrar		2	Celular	Samsung	Galaxy A16 4G	399500	3	
☐	 Editar	 Copiar	 Borrar		3	Smartwatch	JD	Jd Capri 1.83	35500	5	
☐	 Editar	 Copiar	 Borrar		4	Auriculares	Logitech G	Gamer G435	126000	10	
☐	 Editar	 Copiar	 Borrar		5	Impresora	Epson	Ecotank L1250	335699	2	
☐	 Editar	 Copiar	 Borrar		7	Monitor	Cooler Master	Monitor Gamer	141300.7	7	

Tabla de productos

La evidencia visual demuestra que la base de datos asignó los identificadores numéricos correlativos de manera automatizada mediante su mecanismo de identidad, resguardando la integridad referencial y completando el ciclo de vida del dato de forma óptima.

5. Modelado de Relaciones y Normalización de Datos (@ManyToOne)

Con el fin de respetar las reglas de normalización de bases de datos relacionales y garantizar la integridad referencial del sistema, se reemplazó el atributo plano de tipo `String` que almacenaba textualmente la categoría en la clase `Producto`. En su lugar, se estableció una relación dirigida a nivel de objetos y fuertemente tipada mediante las anotaciones nativas de la especificación Jakarta Persistence:

- **@ManyToOne:** Define conceptualmente que múltiples registros de la tabla de productos pueden estar asociados de forma unívoca a una única categoría preexistente en el sistema.
- **@JoinColumn(name = "categoria_id"):** Declara formalmente la creación de una restricción de clave foránea (FK) en la tabla física de MySQL. Esta directiva enlaza de forma automática la columna local con la clave primaria (`id`) de la entidad `Categoria`.

Para validar esta arquitectura en la etapa de inicialización del sistema y agilizar los testeos, se re-configuró el componente de precarga automatizada de datos de prueba mediante la interfaz `CommandLineRunner`, marcado como un `@Bean` administrado en la clase principal del proyecto. Este método inspecciona de forma condicional las tablas mediante las llamadas a los servicios correspondientes; si se detecta que las mismas se encuentran vacías, inyecta en cascada un set homogéneo de categorías y productos enlazados entre sí.

Debido a que la directiva `ddl-auto=update` no elimina estructuras previas de forma automática para prevenir la pérdida de datos, se procedió a realizar un vaciado limpio de las tablas preexistentes. Esto permitió que, en el siguiente inicio de la aplicación, el motor de persistencia generara la nueva estructura optimizada desde cero. En la auditoría de la tabla `producto` se constató la correcta creación de la columna indexada `categoria_id`, verificando que el sistema asignó de manera exacta los identificadores numéricos correspondientes a cada recurso enlazado en el inicio:

				id	descripcion	nombre
☐		Editar		Copiar		Borrar
				1	Dispositivos móviles inteligentes para comunicació...	celular
☐		Editar		Copiar		Borrar
				2	Relojes inteligentes con funciones de monitoreo de...	smartwatch
☐		Editar		Copiar		Borrar
				3	Equipos de impresión, escaneo y copiado de documen...	impresora
☐		Editar		Copiar		Borrar
				4	Dispositivos de audio personales para escuchar mús...	auricular

Tabla categoria

←T→				id	marca	nombre	precio	stock	categoria_id			
☐		Editar		Copiar		Borrar	1	Xiaomi	Redmi Note 10	250000	5	1
☐		Editar		Copiar		Borrar	2	Samsung	Galaxy A16 4G	399500	3	1
☐		Editar		Copiar		Borrar	3	JD	Jd Capri 1.83	40500	3	2
☐		Editar		Copiar		Borrar	4	Logitech G	Gamer G435	126000	10	4
☐		Editar		Copiar		Borrar	5	Epson	Ecotank L1250	335699	2	3

Tabla productos

La evidencia visual confirma el correcto funcionamiento del motor de persistencia Hibernate al resolver los objetos enlazados en memoria y traducirlos en registros con integridad referencial completa dentro de la base de datos local.

6. Ampliación de la Capa de Consultas y Endpoints Avanzados (RESTful API)

Para consolidar el módulo de productos y dotar a la aplicación de las capacidades dinámicas que requiere un comercio electrónico real, se procedió a expandir la capa de persistencia mediante el diseño de consultas avanzadas. En esta fase, se explotaron en paralelo las dos metodologías principales que provee el framework Spring Data JPA: la derivación automática de consultas por nombre de método (**Query Methods**) y la declaración de consultas orientadas a objetos personalizadas (**JPQL**).

Esta infraestructura expande el abanico de operaciones lógicas y comerciales del sistema a través de los siguientes criterios estratégicos implementados en el repositorio:

- **Filtrado Dinámico por Rango de Precios (`findByPrecioBetween`):** Permite acotar las búsquedas del catálogo web abstrayendo una sentencia SQL con operadores **BETWEEN**. En la capa de servicios, se incorporó lógica defensiva para validar que los valores numéricos mínimos y máximos ingresados sean consistentes (impidiendo rangos negativos o que el mínimo supere al máximo) antes de realizar la petición al motor.
- **Alertas de Control de Stock Mínimo (`findByStockLessThan`):** Provee una función crítica orientada al panel de administración del negocio, aislando de forma predictiva aquellos artículos cuya disponibilidad física en los almacenes se encuentra por debajo de un umbral determinado, facilitando la gestión de reposición de mercadería.
- **Normalización de Búsquedas por Marca (`findByMarcaIgnoreCase`):** Optimiza la experiencia del usuario final al mitigar la sensibilidad a mayúsculas y minúsculas

(*case-insensitivity*) en el motor MySQL. Esto asegura que cadenas como "xiaomi", "Xiaomi" o "XIAOMI" resuelvan exactamente hacia el mismo conjunto de registros.

- **Subconsultas Estadísticas de Agregación en JPQL (*obtenerMasCaros*):** Se diseñó una consulta personalizada que ejecuta una subconsulta matemática para aislar el valor máximo de la columna precio mediante la función `MAX()`. De esta manera, el sistema devuelve de forma exacta el o los artículos de mayor valor monetario sin necesidad de ordenar y sobrecargar la memoria RAM de la aplicación.

Toda esta lógica de negocio fue expuesta hacia la red mediante la refactorización integral del componente `ProductoController`. Se estructuró un diseño puramente RESTful utilizando de forma correcta la semántica de las anotaciones de Spring:

- **@PathVariable:** Empleado para rutas fijas y unívocas, tales como la búsqueda directa por marcas o categorías específicas integradas en el patrón de la URL.
- **@RequestParam:** Utilizado como la mejor práctica para inyectar filtros opcionales u operaciones de ordenamiento (como los límites de precio `?min=X&max=Y`), disociando las variables de filtrado de la estructura troncal del recurso.

Finalmente, el controlador fue blindado mediante bloques de control `try-catch` para interceptar las excepciones personalizadas del sistema. Esto garantiza que ante la solicitud de un recurso inexistente o una operación inválida, la API no sufra una caída catastrófica, sino que responda de manera predecible utilizando los códigos de estado del protocolo HTTP estandarizados (200 OK, 201 CREATED, 404 NOT FOUND), alcanzando un estándar de desarrollo profesional y escalable.

Validación de Datos en el Modelo de Dominio

Con el objetivo de desacoplar la lógica de control de datos de la capa de servicios (`Service`) y robustecer la integridad de la información procesada por la API, se migró el esquema de validaciones imperativas previas hacia un modelo declarativo estandarizado mediante **Jakarta Bean Validation** (utilizando **Hibernate Validator** como el proveedor de referencia subyacente).

Esta reingeniería de software permite centralizar las restricciones directamente en el modelo de dominio (las entidades de base de datos), simplificando el mantenimiento del código mediante anotaciones específicas:

- **@NotBlank:** Aplicada en los atributos `nombre` de las clases `Producto` y `Categoria`. Restringe de manera estricta que las cadenas de texto ingresadas contengan caracteres válidos, bloqueando cadenas vacías o compuestas exclusivamente por espacios en blanco. Adicionalmente, se configuraron mensajes de error personalizados (`message = "..."`) para una retroalimentación precisa hacia el cliente.
- **@Positive:** Utilizada en el campo `precio` de `Producto`, garantizando que el valor monetario de un artículo sea estrictamente superior a cero ($\$ > 0\$$), impidiendo distorsiones financieras en las transacciones de compraventa.
- **@PositiveOrZero:** Implementada sobre el atributo `stock` de `Producto`, permitiendo la existencia de artículos con disponibilidad nula (sin existencias físicas, valor 0) pero bloqueando de forma categórica la inserción de inventarios negativos en los almacenes del motor relacional.

```
2026-06-20T19:30:52.132-03:00 WARN 8244 --- [ecommerce] [nio-8080-exec-3] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for argument [0] in public org.springframework.http.ResponseEntity<com.techlab.ecommerce.model.Producto> com.techlab.ecommerce.controller.ProductoController.createProducto(com.techlab.ecommerce.model.Producto): [Field error in object 'producto' on field 'precio': rejected value [-1.0]; codes [Positive.producto.precio,Positive.precio,Positive.double,Positive]; arguments [org.springframework.context.support.DefaultMessageSourceResolvable: codes [producto.precio,precio]; arguments []; default message [precio]]; default message [El precio debe ser mayor que cero]] ]
```

Mensaje mostrado si se ingresa un precio menor a cero

```
2026-06-20T19:31:29.496-03:00 WARN 8244 --- [ecommerce] [nio-8080-exec-4] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for argument [0] in public org.springframework.http.ResponseEntity<com.techlab.ecommerce.model.Producto> com.techlab.ecommerce.controller.ProductoController.createProducto(com.techlab.ecommerce.model.Producto): [Field error in object 'producto' on field 'nombre': rejected value []; codes [NotBlank.producto.nombre,NotBlank.nombre,NotBlank.java.lang.String,NotBlank]; arguments [org.springframework.context.support.DefaultMessageSourceResolvable: codes [producto.nombre,nombre]; arguments []; default message [nombre]]; default message [El nombre del producto no puede estar vacío]] ]
```

Mensaje mostrado si se ingresa un nombre vacío o nulo

Para activar este mecanismo de interceptación automática en el flujo de peticiones web de la arquitectura en capas, se incorporó la anotación **@Valid** en los parámetros de los métodos mutadores (POST y PUT) dentro de `ProductoController` y `CategoriaController`.

Cuando un cliente envía un objeto JSON para registrar o actualizar un recurso, el framework intercepta el cuerpo de la solicitud (**@RequestBody**) y somete el objeto al conjunto de restricciones declaradas en la entidad **antes de que la petición acceda a los métodos de la capa de servicios**. Si alguna regla es vulnerada, el framework detiene la ejecución inmediatamente, previniendo estados inconsistentes de forma nativa e impidiendo la persistencia de datos corruptos en el servidor de bases de datos.

Arquitectura y Manejo Global de Errores

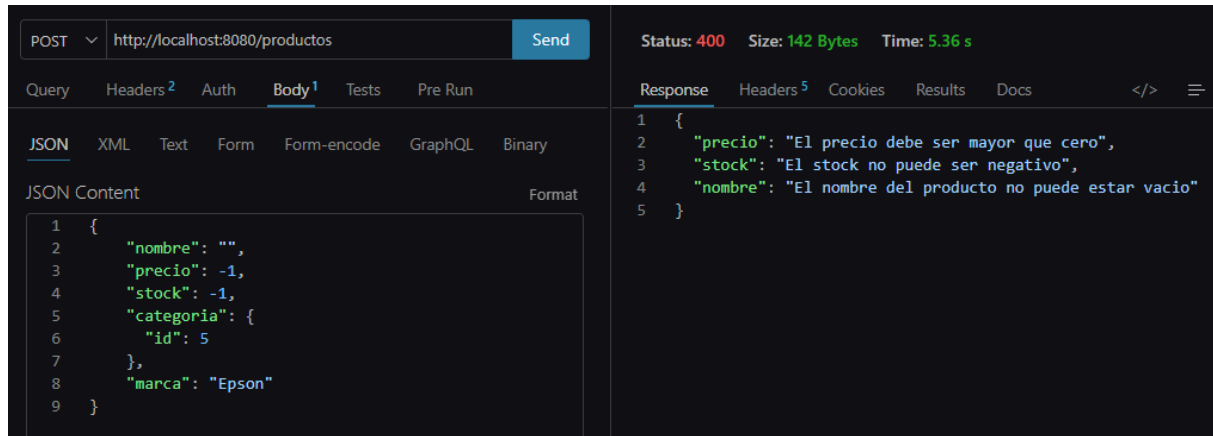
A fin de optimizar el diseño arquitectónico de la aplicación y erradicar la duplicación de estructuras de control, se implementó un mecanismo de gestión centralizada de errores basado en el patrón *Interceptor*. Mediante la incorporación de la clase `GlobalExceptionHandler`, anotada con `@ControllerAdvice`, se logró desacoplar por completo la captura de anomalías de los controladores REST nativos.

Esta reingeniería de la infraestructura del backend elimina la necesidad de estructurar bloques `try-catch` redundantes en cada uno de los endpoints de `ProductoController` y `CategoriaController`. Cuando una regla de negocio es vulnerada en la capa de servicios (como la solicitud de un identificador inexistente), el hilo de ejecución propaga la excepción hacia la periferia de la aplicación, donde es interceptada de forma automática por este componente centralizado.

El manejador global unifica los criterios de respuesta a través de métodos especializados gobernados por la anotación `@ExceptionHandler`:

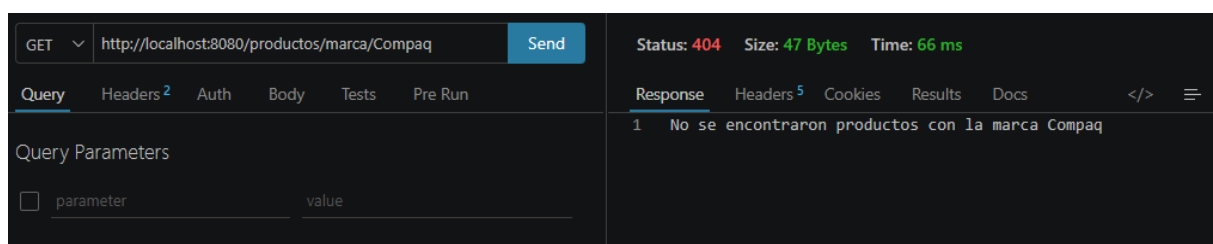
- **Gestión de Entidades No Encontradas:** Al capturar excepciones personalizadas como `ProductoNoEncontradoException` o `CategoriaNoEncontradaException`, el interceptor desvía de forma automática el flujo y construye una respuesta con el estado semántico preciso `404 NOT FOUND`, adjuntando únicamente el mensaje contextual descriptivo en el cuerpo del mensaje.
- **Control de Restricciones de Negocio:** Errores transaccionales o de formato (por ejemplo, `CategoriaNombreInvalidoException`) son encapsulados y devueltos bajo el código de estado `400 BAD REQUEST`, normalizando el comportamiento de la API frente a operaciones inconsistentes.
- **Procesamiento de Errores de Validación Estructural:** Para dar soporte al motor de validación declarativa implementado en la fase previa, se diseñó un método específico dedicado a interceptar la excepción nativa del framework `MethodArgumentNotValidException`. Este componente extrae de forma iterativa cada uno de los campos que han fallado los filtros de restricción (como nombres vacíos o valores numéricos negativos) y construye un mapa estructurado clave-valor en formato JSON, el cual es despachado con un estado `400 BAD REQUEST`.

A modo de ilustración, en la Figura siguiente se observa una prueba de integración realizada desde el cliente HTTP *Thunder Client*. Al enviar intencionalmente un cuerpo JSON con el atributo `nombre` vacío y valores negativos en `precio` y `stock`, el interceptor global detiene la petición y estructura una respuesta limpia con los mensajes de error configurados en el modelo:

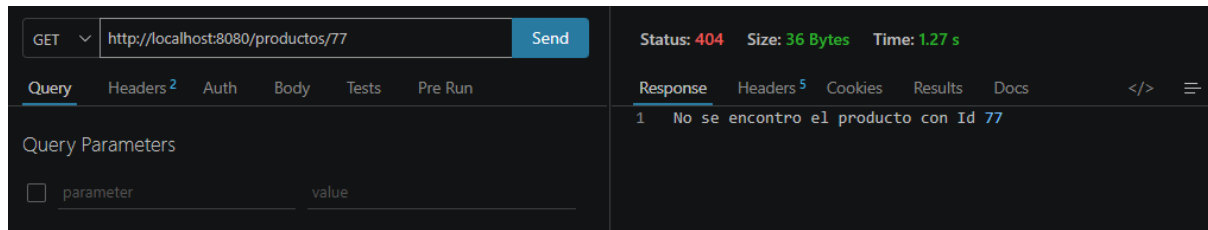


Interceptación y formateo de `MethodArgumentNotValidException` con respuesta estructurada HTTP 400.

Por otra parte, en las Figuras a continuación se evidencian las pruebas de integración correspondientes a la búsqueda de recursos inexistentes o parámetros sin coincidencias en el motor relacional. Al intentar recuperar un artículo mediante un identificador numérico aleatorio (Id: 77) o al filtrar por una firma comercial que no posee registros en la base de datos (Marca: Compaq), el flujo del sistema propaga las excepciones `ProductoNoEncontradoException` y la lógica de validación de colecciones vacías, respectivamente. Como se constata en las respuestas del cliente HTTP, el interceptor global unifica la salida del servidor homologando el código de estado a un estricto HTTP 404 NOT FOUND y transmitiendo un mensaje en texto plano completamente dinámico y contextualizado para orientar al consumidor de la API:



Interceptación de ausencia de registros por marca con respuesta dinámica HTTP 404.



Interceptación de `ProductoNoEncontradoException` por identificador único con respuesta HTTP 404.

Como resultado de esta refactorización, se obtuvo un desacoplamiento absoluto de responsabilidades: los controladores se reducen a meros despachadores de peticiones y respuestas lineales, delegando el tratamiento de fallos a una única clase especializada. Esto incrementa exponencialmente la legibilidad, la robustez ante caídas imprevistas y proporciona un contrato de interfaz JSON limpio, predecible y de fácil consumo para cualquier tecnología de cliente o Frontend.

Módulo de Gestión de Pedidos y Carrito de Compras

En esta sección se detalla la arquitectura, lógica de negocio y endpoints desarrollados para el procesamiento de pedidos, el cálculo automatizado de costos y la sincronización en tiempo real del stock físico de los productos.

1. Modelo de Datos y Relaciones (ORM)

Para implementar el carrito y el historial de compras de manera relacional, se diseñaron dos entidades principales interconectadas:

- **Pedido (Tabla pedidos):** Representa la cabecera de la orden. Almacena el identificador único (`id`), la marca de tiempo de la operación (`fecha_alta` mediante `LocalDateTime`) y el importe final acumulado (`total`). Se vincula con las líneas mediante una relación `@OneToMany` configurada con `CascadeType.ALL` y `orphanRemoval = true`, lo que garantiza que cualquier operación de persistencia o eliminación sobre el pedido impacte automáticamente en sus componentes hijos.
- **LineaPedido (Tabla lineas_pedidos):** Representa el desglose del carrito (cada ítem individual). Registra la `cantidad` solicitada y el `subtotal` parcial calculado. Se conecta con el catálogo mediante una relación `@ManyToOne` hacia la entidad `Producto` (`producto_id`), asegurando la integridad referencial.

2. Lógica de Negocio y Robustez del Sistema

La capa de servicio concentra las reglas del negocio de la aplicación, priorizando la consistencia de los datos bajo dos pilares fundamentales:

- A. Operaciones Atómicas y Control de Concurrency:** Tanto el flujo de alta como el de cancelación de pedidos se encuentran bajo la anotación `@Transactional` de Spring. Esto actúa como un escudo de seguridad ("todo o nada"): si ocurre algún fallo o excepción en medio del bucle de procesamiento, el framework ejecuta un *Rollback* automático, revirtiendo cualquier modificación previa en las tablas y evitando estados inconsistentes o "datos fantasma" en la base de datos.
- B. Algoritmo de Creación de Pedidos:** Cuando un cliente envía una estructura de pedido desde el cliente HTTP, el servicio ejecuta el siguiente flujo secuencial automatizado:
1. **Validación de Existencia:** Se intercepta el ID del producto recibido y se consulta a la base de datos a través de `ProductoRepository`. Si el producto no existe en el catálogo físico, se interrumpe el flujo lanzando un `ProductoNoEncontradoException`.
 2. **Control Crítico de Stock:** Se evalúa si el stock disponible en la base de datos cubre la cantidad solicitada. De no haber existencias suficientes, se dispara una excepción de negocio controlada: `StockInsuficienteException`.
 3. **Sincronización del Inventario:** Si la validación es exitosa, se descuenta la cantidad comprada del stock físico del producto y se persiste inmediatamente el nuevo inventario en la tabla `producto`.
 4. **Cálculo Blindado de Precios:** Para evitar alteraciones maliciosas desde el cliente, el sistema descarta los precios enviados en el JSON y utiliza exclusivamente el precio real almacenado en el backend. Calcula el `subtotal` de la línea (`precio * cantidad`) y lo acumula en el `totalGeneral`.
 5. **Persistencia Final:** Se asocian las líneas definitivas al pedido, se setea el monto total calculado y se guarda el registro de forma unificada.
- C. Algoritmo de Cancelación:** Al solicitar la baja de un pedido por su ID, el servicio recupera la orden y recorre cada una de sus líneas antes de efectuar el borrado. De forma automatizada, **restituye las cantidades compradas sumándolas nuevamente al stock actual de cada producto**, asegurando que el inventario físico vuelva a su estado original antes de ejecutar el `DELETE` de la orden en MySQL.

3. Arquitectura de Control de Errores Centralizada

Para brindar una experiencia limpia y estructurada ante fallos del cliente, se integraron las excepciones del módulo al sistema de captura global `GlobalExceptionHandler`:

- **StockInsuficienteException:** Cuando un pedido supera las existencias, la excepción es capturada en la capa intermedia, evitando un código de error de servidor interno (500 Internal Server Error). El manejador retorna un estado controlado **400 Bad Request** acompañado del mensaje descriptivo exacto (ej. *"Stock insuficiente para el producto 'Xiaomi'. Stock disponible: 3, solicitado: 5"*).

Catálogo de Endpoints de la API REST (Diccionario de Rutas)

A continuación, presento la especificación detallada de todas las rutas HTTP expuestas por la aplicación. La API maneja todas sus respuestas en formato **JSON** y utiliza los códigos de estado HTTP estándar para indicar el éxito o fallo de las operaciones.

Módulo de Categorías (/categorias)

Método HTTP	Endpoint	Descripción	Código Exitoso	Parámetros / Body
GET	/categorias	Recupera la lista completa de categorías registradas.	200 OK	Ninguno
GET	/categorias/{id}	Busca y retorna una categoría específica mediante su ID.	200 OK	id (en la URL)
POST	/categorias	Registra una nueva categoría en el sistema.	201 CREATED	Body: Objeto Categoria (@Valid)
PUT	/categorias/{id}	Modifica los datos de una categoría existente por su ID.	200 OK	id (en la URL) + Body: Categoria

Módulo de Productos (/productos)

Es el componente más robusto de la API. Cuenta con endpoints para el ABM tradicional y múltiples rutas de filtrado de datos para optimizar las búsquedas del frontend.

Método HTTP	Endpoint	Descripción	Código Exitoso	Parámetros / Body
GET	/productos	Obtiene la lista completa de productos con sus detalles.	200 OK	Ninguno
GET	/productos/{id}	Busca un producto por su identificador único.	200 OK	id (en la URL)
GET	/productos/nombre/{nombre}	Filtra productos que coincidan con el nombre enviado.	200 OK	nombre (en la URL)
GET	/productos/categoria/{categoria}	Recupera los productos pertenecientes a una categoría.	200 OK	categoria (en la URL)
GET	/productos/filtro-precio	Filtra productos dentro de un rango de precios mínimo y máximo.	200 OK	Parámetros Query: ?min=X&max=Y
GET	/productos/stock-bajo/{limite}	Alerta sobre productos cuyo stock sea inferior al límite dado.	200 OK	limite (en la URL)
GET	/productos/marca/{marca}	Recupera todos los productos de una marca específica.	200 OK	marca (en la URL)
GET	/productos/mas-caro	Retorna el o los productos con el precio más alto del catálogo.	200 OK	Ninguno
POST	/productos	Crea y persiste un nuevo producto con su descripción y URL de imagen.	201 CREATED	Body: Objeto Producto (@Valid)
PUT	/productos/{id}	Actualiza la información completa de un producto por su ID.	200 OK	id (en la URL) + Body: Producto
DELETE	/productos/{id}	Elimina físicamente un producto del catálogo por su ID.	200 OK	id (en la URL)

Módulo de Pedidos (/pedidos)

Encargado de procesar las compras del carrito, impactar el total general y actualizar el stock de inventario de forma segura.

Método HTTP	Endpoint	Descripción	Código Exitoso	Parámetros / Body
GET	/pedidos	Retorna el historial de pedidos con el desglose de sus líneas.	200 OK	Ninguno
POST	/pedidos	Procesa un carrito de compras, valida stock y calcula subtotales.	201 CREATED	Body: Objeto Pedido con sus líneas
DELETE	/pedidos/{id}	Cancela un pedido por ID y restituye las cantidades al stock de productos.	200 OK	id (en la URL)

Nota Técnica: Manejo de CORS configurado

Los controladores de **Productos** y **Pedidos** exponen la anotación `@CrossOrigin` para habilitar el intercambio de recursos de origen cruzado. Esto permitirá que la aplicación Frontend (desarrollada en React o HTML/JS local) pueda consumir libremente estos endpoints sin bloqueos de seguridad del navegador.

Es importante mencionar que para interactuar correctamente con el endpoint de alta de pedidos (POST /pedidos), el cliente frontend o la herramienta de pruebas (*Thunder Client*) debe enviar una estructura JSON simplificada. El backend está diseñado de forma robusta para procesar únicamente los datos necesarios y mitigar manipulaciones maliciosas:

```
{
  "lineas": [
    {
      "producto": { "id": 1, "precio": 0.0, "stock": 0 },
      "cantidad": 2
    },
    {
      "producto": { "id": 4, "precio": 0.0, "stock": 0 },
      "cantidad": 1
    }
  ]
}
```

Estructura estándar del payload JSON (Request Body) requerido para la creación exitosa de un pedido

Conclusiones Finales

El desarrollo del presente proyecto ha permitido consolidar una evolución tecnológica sustancial en el diseño de software orientado a objetos, marcando una transición crítica desde una aplicación inicial basada puramente en la consola de comandos hacia una arquitectura empresarial moderna, robusta y escalable implementada con el framework **Spring Boot**.

A lo largo de este proceso incremental, se destacan las siguientes conclusiones técnicas:

- **Desacoplamiento y Arquitectura en Capas:** La migración hacia el patrón arquitectónico REST permitió separar estrictamente las responsabilidades del sistema a través de controladores, servicios y repositorios (JPA). Esto garantiza que la lógica de negocio permanezca aislada de los mecanismos de persistencia y de la capa de presentación.
- **Integridad de los Datos y Lógica de Negocio Centralizada:** Se logró blindar el sistema contra manipulaciones maliciosas o erróneas desde el cliente (como intentos de alteración de precios o stock mediante payloads JSON). Al centralizar el cálculo financiero de subtotales y la validación física de inventarios en el servidor (**PedidoService**), la base de datos relacional MySQL mantiene una consistencia absoluta.

Como paso subsiguiente, el backend desarrollado se establece como una plataforma madura, versátil y completamente blindada mediante políticas de intercambio de recursos de origen cruzado (@CrossOrigin). Al estar diseñada bajo los estándares puros de una API REST que transacciona datos en formato JSON, la interfaz de usuario puede ser construida e integrada utilizando múltiples tecnologías del ecosistema web. En una siguiente fase, se puede optar, por ejemplo, por una arquitectura ágil y eficiente basada en tecnologías nativas (**HTML5, CSS y JavaScript**),

consumiendo los endpoints necesarios mediante la API Fetch para garantizar una experiencia de usuario fluida, reactiva y liviana.

A nivel estrictamente personal, el recorrido de este trayecto académico ha resultado ser una experiencia sumamente enriquecedora e interesante. Los contenidos y desafíos planteados a lo largo del curso no solo expandieron mis horizontes teóricos, sino que me dotaron de herramientas prácticas y conocimientos técnicos de vanguardia que son altamente demandados en el mercado actual.

El haber transformado un programa básico de consola en un backend robusto con persistencia real y arquitectura profesional representa un hito fundamental en mi formación, consolidando bases sólidas que, sin lugar a dudas, potenciarán mi crecimiento y competitividad en el ámbito profesional del desarrollo de software.

Anexo

Modificación del Modelo de Datos por Requerimientos de Interfaz (Frontend)

Durante la fase final del desarrollo del backend y la planificación de la interfaz de usuario (Frontend), se identificó la necesidad de enriquecer la experiencia visual del catálogo de productos. El modelo inicial solo contemplaba datos operativos básicos (nombre, precio, marca, stock). Para transformar el sistema en un e-commerce funcional y comercialmente viable, se volvió indispensable asociar una descripción detallada a cada artículo y un recurso multimedia (imagen).

Para implementar esto de forma segura y sin afectar la integridad de las órdenes de compra y registros existentes en la base de datos, entró en acción la propiedad `spring.jpa.hibernate.ddl-auto=update` que ya se encontraba configurada en el archivo `application.properties`. Esto permitió que Hibernate ejecutara una migración automática (`ALTER TABLE`) en MySQL, extendiendo el esquema de la tabla de manera transparente.

A continuación, se detalla el esquema final de la tabla `productos` con las nuevas columnas incorporadas:

Columna	Tipo de Datos (MySQL)	Restricción	Descripción
id	INT	PRIMARY KEY , AUTO_INCREMENT	Identificador único del producto.
nombre	VARCHAR(255)	NOT NULL	Nombre comercial del artículo.
marca	VARCHAR(255)	NOT NULL	Fabricante o marca del producto.
precio	DOUBLE	NOT NULL	Precio de venta unitario oficial.
stock	INT	NOT NULL	Cantidad física disponible en inventario.
descripcion	VARCHAR(255)	NULL	Detalle y especificaciones comerciales del producto.
url_imagen	VARCHAR(500)	NULL	Enlace directo puro (https://...)
categoria_id	INT	FOREIGN KEY	Vinculación relacional con la tabla de categorías.

		id	descripcion	marca	nombre	precio	stock	url_imagen	categoria_id
☐	 Editar  Copiar  Borrar	1	La impresora Epson Ecotank L1250 es compacta y lig...	Epson	Impresora Ecotank L1250	335.699	5	https://i.ibb.co/G4SHTXGH/a51ba38cc959.jpg	
☐	 Editar  Copiar  Borrar	2	El celular Samsung Galaxy A16 cuenta con una impre...	Samsung	Galaxy A16 4G	399500	3	https://i.ibb.co/m5pScXv5/4941eefb9b41.jpg	
☐	 Editar  Copiar  Borrar	3	Con el Smartwatch JD, puedes monitorear tu salud l...	JD	Jd Capri 1.83	40500	3	https://i.ibb.co/k68mMnKH/8b20da0585e3.png	
☐	 Editar  Copiar  Borrar	4	Con una construcción ligera, estos auriculares ina...	Logitech G	Gamer G435	126000	10	https://i.ibb.co/k6chhKHG/7607a6bdfd5e.png	
☐	 Editar  Copiar  Borrar	5	El modo de carga inteligente durante la noche, jun...	TCL	TCL 50 Space Gray	921399	2	https://i.ibb.co/YBXhR2sP/62e7607041c9.png	

Vista física de la tabla productos en phpMyAdmin reflejando la persistencia de las nuevas columnas y los enlaces multimedia verificados.

Arquitectura de Persistencia de Pedidos

En el presente anexo se detalla la justificación técnica de la solución arquitectónica implementada para el procesamiento de transacciones comerciales en el backend, específicamente en lo que respecta al ciclo de vida del carrito de compras y la creación de las entidades Pedido (Order) y Línea de Pedido (OrderLine).

Estrategia de Persistencia y Transaccionalidad

La arquitectura seleccionada delega la gestión del estado dinámico del carrito de compras de forma exclusiva al Frontend (memoria de la aplicación o almacenamiento local del navegador), postergando la interacción con la base de datos relacional hasta la confirmación explícita de la transacción por parte del usuario (operación de checkout).

Ventajas Arquitectónicas de la Solución Implementada:

- **Optimización de Recursos del Servidor:** Evita la sobrecarga de operaciones I/O concurrentes en la base de datos por cada interacción del usuario en la interfaz.
- **Garantía de Integridad (Transaccionalidad):** El procesamiento se ejecuta bajo un contexto aislado mediante la anotación `@Transactional` en la capa de servicios, asegurando que se cumplan las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).
- **Reducción de Registros Huérfanos:** Se elimina de raíz la generación de "carritos abandonados" en almacenamiento persistente, optimizando el espacio en el motor de base de datos.

Líneas de Desarrollo Futuro

Para las siguientes fases evolutivas del Sistema de Gestión TechLab, se proyecta la migración hacia un modelo multidimensional mediante la incorporación de la entidad **Cliente (Customer)**. Actualmente, el sistema se enfoca con éxito en la consistencia transaccional y el control estricto de stock entre las entidades **Pedido** y **LíneaPedido**.

Justificación del alcance actual:

- Se priorizó mitigar el acoplamiento y el "efecto dominó" en el flujo transaccional inmediato. Exigir la existencia previa de un registro de cliente obligaría a reestructurar la capa de lógica, controladores y repositorios, alterando la eficiencia de las pruebas actuales.
- La inclusión de la entidad Cliente permitirá, en una Fase Posterior, implementar un sistema de autenticación de usuarios (Spring Security), persistencia de carritos multidispositivo, trazabilidad del historial de compras y gestión de perfiles comerciales, sin comprometer la robustez del motor de pedidos ya consolidado.